

Selenium Installation and Troubleshooting Assignment

Python • WebDriver • Browser Automation • Diagnostic Skills

Names	Tuheena Bose, Arunavo Bhowal, Debapriya Mistry	Roll No.	08, 38, 47
Section	A, A, A	Date	27/08/2026

Part B — Technical Investigation

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

A: Selenium is an open-source automation framework primarily used to automate web browsers. It allows programs to control browsers such as Chrome, Firefox, and Edge to perform actions like clicking buttons, entering text, navigating pages, and verifying web-page behavior.

Applications Selenium is designed to automate

Selenium is mainly designed for web-based applications accessed through a browser, especially for:

- Automated functional and regression testing
- End-to-end testing of websites
- Cross-browser testing
- Repetitive browser-based tasks

Suitable use cases

1. **Testing an e-commerce website** — automatically test login, product search, adding items to a cart, and checkout.
2. **Testing a web-based banking application** — verify that login, account navigation, and transaction workflows work correctly.

Unsuitable use cases

1. **Automating desktop applications** such as Microsoft Word or Windows Calculator, because Selenium is designed for web browsers.

2. **Automating mobile applications directly** (Android/iOS native apps), since Selenium WebDriver is primarily intended for browser-based automation. Tools such as Appium are more appropriate for native mobile apps.

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

A:

Feature	Selenium WebDriver	Selenium IDE	Selenium Grid
Core Purpose	Writing robust, code-based test automation scripts.	Rapid prototyping via record-and-playback features.	Running tests in parallel across multiple systems.
Coding Required	High. Requires programming knowledge.	None. It is a completely codeless tool.	Dependent on underlying WebDriver scripts.
Interface	No UI; it functions as an API library.	GUI tool installed as a browser extension.	Console/Dashboard for network infrastructure.
Execution Speed	Fast, as it talks directly to the browser.	Slower; relies on browser plugin execution.	Dependent on network node speed and infrastructure.
Parallel Execution	Requires external frameworks (e.g., TestNG).	Very limited/Not natively designed for it.	Natively built for cross-environment parallel tests.
Language Support	Supports Java, Python, C#, Ruby, JavaScript, etc.	Uses its own language format called Selenese.	Language independent (manages the execution nodes).

Architecture	Client-Server using browser-specific native drivers.	Browser add-on framework.	Hub-and-Node distributed network architecture.
Complex Logic	Easily handles loops, conditionals, and data driving.	Difficult to implement complex programming logic.	Focuses on infrastructure management, not test logic.

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

A: For Selenium, a virtual environment is recommended because it creates an isolated Python environment for our project.

- **Why use a virtual environment?**

It allows us to install Selenium and its required packages separately from other Python projects. This prevents dependency conflicts and makes it easier to manage the project's package versions.

- **What happens with global installation?**

Installing packages globally can cause version conflicts. For example, one project may require a particular Selenium or Python package version, while another project requires a different version. Updating a globally installed package for one project could therefore break another project.

Example:

If Project A requires Selenium 4.47.0 and Project B depends on a different compatible setup, installing everything globally can create conflicts. With virtual environments, each project can have its own independent package setup.

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

A: Selenium Manager is a built-in tool in modern Selenium that automatically manages browser drivers such as ChromeDriver, GeckoDriver, and EdgeDriver.

How it changed driver setup

Previously, users generally had to:

- Download the correct browser driver manually.
- Add the driver to the system PATH, or specify its location in the Selenium code.

With recent Selenium versions, Selenium Manager can automatically detect the installed browser and obtain/use the appropriate driver. This means we can usually write:

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

and Selenium Manager will handle the driver setup automatically.

Part C — Installation Challenges to Solve

C1 — pip is not recognized

Check whether Python is installed:

```
python --version
```

Try running pip through Python:

```
python -m pip --version
```

If this works, use:

```
python -m pip install selenium
```

```
PS C:\Users\Tuheena Bose> python --version
Python 3.10.0
PS C:\Users\Tuheena Bose> python -m pip --version
pip 25.3 from C:\Users\Tuheena Bose\AppData\Local\Programs\Python\Python310\lib\site-packages\pip (python 3.10)
PS C:\Users\Tuheena Bose> ^C
PS C:\Users\Tuheena Bose> python -m pip install selenium
Requirement already satisfied: selenium in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (4.47.0)
Requirement already satisfied: certifi<=2026.2.25 in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (from selenium) (2026.7.22)
Requirement already satisfied: trio<1.0,>=0.31.0 in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (from selenium) (0.34.0)
Requirement already satisfied: trio-websocket<1.0,>=0.12.2 in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (from selenium) (0.12.2)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (from selenium) (4.15.0)
Requirement already satisfied: urllib3<3.0,>=2.6.3 in c:\users\tuheena bose\appdata\local\programs\python\python310\lib\site-packages (from urllib3)
```

If Python itself is not recognized, Python may not be installed correctly or its installation directory may not be in the PATH. Reinstall Python and make sure "Add Python to PATH" is selected.

C2 — Package installed, import fails

The most common reason is that Selenium was installed in a different Python environment from the one being used by the IDE. For example, pip may install Selenium in the system Python environment while the IDE is using a virtual environment.

Steps to Troubleshoot

1. Check which Python interpreter is being used

Run:

```
python --version
```

Then check the Python location:

Windows:

where python

Linux/macOS:

which python

2. Check which pip is being used

```
python -m pip --version
```

This shows the location where packages are being installed.

3. Check whether Selenium is installed

```
python -m pip show selenium
```

If Selenium is not found, install it using:

```
python -m pip install selenium
```

Using `python -m pip` is recommended because it makes sure pip belongs to the Python interpreter you are using.

4. Check the IDE interpreter

In the IDE, open the Python Interpreter settings and check which Python interpreter is selected.

5. Install Selenium in the active virtual environment

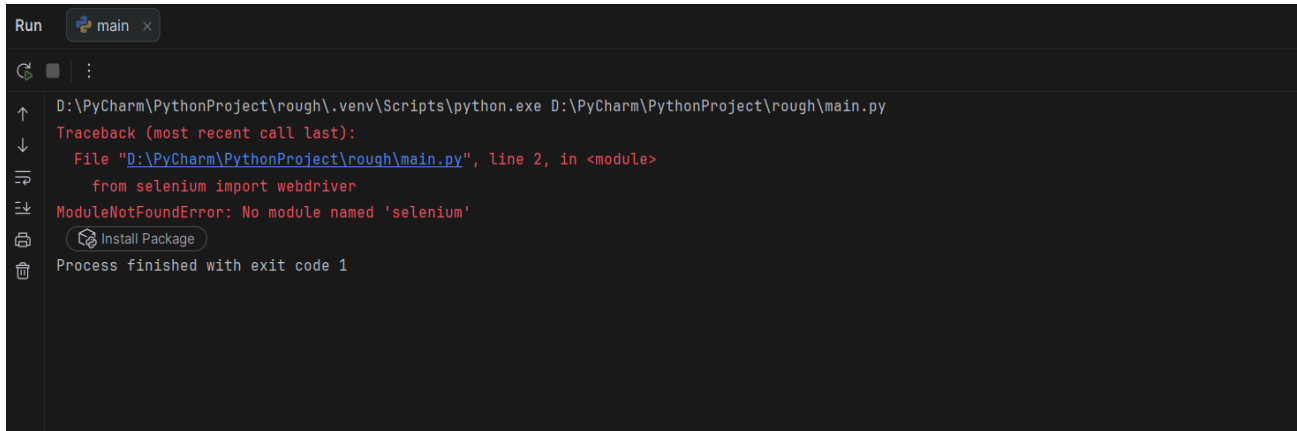
Activate the virtual environment and run:

```
python -m pip install selenium
```

Then test the import:

```
import selenium
```

```
print(selenium.__version__)
```



```
Run main x
D:\PyCharm\PythonProject\rough\.venv\Scripts\python.exe D:\PyCharm\PythonProject\rough\main.py
Traceback (most recent call last):
  File "D:\PyCharm\PythonProject\rough\main.py", line 2, in <module>
    from selenium import webdriver
ModuleNotFoundError: No module named 'selenium'
Install Package
Process finished with exit code 1
```

Reflection

R2. What would you change in your setup to make future Selenium projects easier to reproduce?

A: To make future Selenium projects easier to reproduce, we would use a separate virtual environment for each project and maintain a requirements.txt file containing the required package versions. We would also use a consistent Python version and rely on Selenium Manager for automatic browser-driver management. This ensures that the same dependencies and setup can be recreated easily on another computer.

R3. Which source was most useful, and how did you verify that its advice applied to your Selenium version?

A: The official Selenium documentation was the most useful source because it provides reliable and up-to-date information about Selenium installation and Python support.

I verified that its advice applied to my Selenium version by checking the installed version with:

```
python -m pip show selenium
```

or:

```
import selenium
```

```
print(selenium.__version__)
```

I then compared the installed version with the version and installation instructions given in the official Selenium documentation. This helped ensure that the troubleshooting steps, such as using the correct Python interpreter and installing Selenium with `python -m pip install selenium`, were appropriate for my setup.